Taller práctico: Jenkins con Docker

Integración y despliegue continuo — Ingeniería de Software III

Diego López Camacho — Universidad Industrial de Santander

28 de octubre de 2026

1. Resultado

Las cinco actividades completadas. Al final queda una cadena que funciona sola: un `git push` dispara pruebas en un contenedor Maven, construye una imagen Docker, la somete a un smoke test y solo entonces reemplaza la versión en ejecución.

```
#1  SUCCESS   actividad 3: compilar y probar desde Git
#2  FAILURE   prueba rota a propósito
#3  SUCCESS   corregida
#4  SUCCESS   actividad 4: imagen + smoke test
#5  SUCCESS   actividad 5: despliegue
#6  SUCCESS   version 2.0.0
#7  FAILURE   3.0.0 con un error
#8  SUCCESS   3.0.0 corregida
```

Ocho builds, dos en rojo **a propósito**, y en ninguno de los dos llegó código defectuoso al puerto público. Esa es, en una línea, la razón de ser del taller.

Dos ajustes del entorno

Puertos. La guía usa el 8080 para Jenkins y el 8000 para la app desplegada. En este equipo los dos estaban ocupados por contenedores ajenos a la materia (`mi-backend-api` y `jupyterhub`), así que moví el lado del **host**: Jenkins quedó en **8081** y la app en **8001**. Los puertos internos no se tocaron, y es exactamente lo que prescribe la tabla de problemas del README del material.

Sin navegador. El asistente de instalación de Jenkins se completa normalmente haciendo clic en el navegador. Como trabajo desde terminal, lo hice contra la API REST: desbloqueo con la contraseña inicial, instalación de los plugins sugeridos que faltaban, creación del usuario administrador y `completeInstall`. El resultado verificable es el mismo, y de hecho los puntos de control del taller se comprueban mejor así.

Componente	Dónde quedó
Jenkins	http://localhost:8081
App desplegada	http://localhost:8001
Repositorio de la app	github.com/diegolopezrm/taller-jenkins-app

2. Actividad 1 — Levantar Jenkins

El `docker compose up -d` levanta **dos** contenedores, y entender por qué son dos es lo más importante de esta actividad:

```
  jenkins          Up   0.0.0.0:8081->8080/tcp, 0.0.0.0:50000->50000/tcp
  jenkins-docker   Up   2375-2376/tcp, 0.0.0.0:8001->8000/tcp
```

`jenkins` es el servidor web: trae el **cliente** de Docker, pero no un demonio. `jenkins-docker` es un *Docker-in-Docker*: ahí es donde de verdad se construyen las imágenes y corren los contenedores. Jenkins le habla por TCP con certificados, gracias a las variables `DOCKER_HOST`, `DOCKER_CERT_PATH` y `DOCKER_TLS_VERIFY` del compose.

La prueba de que la conexión funciona es que `docker version` **desde dentro del contenedor de Jenkins** devuelve las dos secciones:

```
$ docker exec jenkins docker version
  Client: Docker Engine - Community
  Server: Docker Engine - Community
    Version:          29.9.2
```

Si solo apareciera `Client`, el cliente estaría ahí pero sin demonio al otro lado, y todas las actividades siguientes fallarían.

Punto de control de la actividad 1 — los tres cumplidos:

- El panel responde `Welcome to Jenkins!`
- `docker version` muestra Client y Server
- `Docker Pipeline v653.v2f2c08eff0ec` aparece activo

La imagen de Jenkins viene de un `Dockerfile` propio que instala el cliente de Docker sobre `jenkins/jenkins:lts-jdk21` y preinstala el plugin `docker-workflow`. De los plugins sugeridos faltaban cuatro —Pipeline, Git, JUnit y Stage View—, que instalé por la API. Quedaron 74 activos.

3. Actividad 2 — El primer pipeline

Un pipeline declarativo con tres etapas y dos de ellas en paralelo:

```
    Saludo                SUCCESS    79 ms
    Info del entorno      SUCCESS   677 ms
    Etapas en paralelo    SUCCESS    73 ms
      A                   SUCCESS    67 ms
      B                   SUCCESS    66 ms
```

La consola confirma el recorrido completo:

```
  Hola desde Taller Jenkins + Docker — build #1
  + whoami
  jenkins
  + docker version
  Client: Docker Engine - Community
  Server: Docker Engine - Community
  Tarea A
  Tarea B
  Pipeline OK ✔
  Finished: SUCCESS
```

Lo que hace útil este pipeline tan simple es la etapa *Info del entorno*: ejecuta `docker version` **desde dentro de un build**, no desde mi terminal. Eso prueba que el agente hereda el acceso al dind, que es la condición para las actividades 4 y 5.

Mini-reto 1 — parámetros. Agregando `parameters { string(name: 'NOMBRE', defaultValue: 'mundo') }` y usando `${params.NOMBRE}`, el saludo pasó de `Hola, mundo` a `Hola, Diego`. Un detalle que cuesta entender la primera vez: el primer build **todavía sale con el valor por defecto**, porque Jenkins registra los parámetros al terminar ese build. Mi intento de lanzarlo con el parámetro antes de tiempo recibió un `HTTP 400`.

Mini-reto 2 — fallo. Agregando una etapa `sh 'exit 1'`:

```
  + exit 1
  Algo falló ✘
  ERROR: script returned exit code 1
  Finished: FAILURE
```

El bloque `post { failure { ... } }` se disparó. Es el mecanismo que en un proyecto real avisaría al equipo.

4. Actividad 3 — El pipeline vive en el repositorio

Aquí cambia algo de fondo: el pipeline deja de estar escrito en Jenkins y pasa a vivir en el repositorio, como un archivo más del proyecto. El job solo guarda *dónde* buscarlo (`Pipeline script from SCM`, rama `*/main`, `Script Path: Jenkinsfile`).

El beneficio es que el pipeline se versiona junto al código: cambia con él, se revisa en los mismos *pull requests* y cualquiera puede reconstruir el proyecto clonando el repositorio.

El `Jenkinsfile` declara un agente Docker, y eso resuelve un problema que no es obvio:

```
agent {
    docker {
        image 'maven:3.9-eclipse-temurin-21'
        args '--network host'
        reuseNode true
    }
}
```

Maven nunca se instala en Jenkins. Se levanta un contenedor por build, se compila dentro y se descarta. En la consola se ve el `docker run` del agente:

```
$ docker run -t -d -u 1000:1000 --network host -w /var/jenkins_home/workspace/actividad3-app
    ... maven:3.9-eclipse-temurin-21 cat
```

Hay dos detalles del `environment` que parecen menores y no lo son. `HOME = "${env.WORKSPACE}"` y `-Dmaven.repo.local=.m2/repository` existen porque el contenedor corre con el uid 1000 sin directorio personal propio: sin eso, Maven falla con *Could not create local repository*. Y `--network host` está para que el contenedor resuelva DNS cuando la red bloquea los servidores públicos.

Resultado del build #1:

```
  Checking out Revision 5cd01a9a... (refs/remotes/origin/main)
  [INFO] Tests run: 4, ... -- in taller.AppTest
  [INFO] Tests run: 2, ... -- in taller.CalculadoraTest
```

```
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
Archiving artifacts
```

Con `Test Result` mostrando 6 pruebas y `taller-app.jar` archivado como artefacto.

Mini-reto. Cambié `assertEquals(5, ...)` por 6, hice `git push` y el build quedó rojo:

```
org.opentest4j.AssertionFailedError: expected: <6> but was: <5>
[ERROR]   CalculadoraTest.sumaDosNumeros:11 expected: <6> but was: <5>
```

Al devolver el valor y volver a subir, verde otra vez.

5. Actividad 4 — Construir la imagen

El reto de esta actividad es entender **dónde vive cada herramienta**. Maven está dentro de su contenedor, pero el cliente de Docker está en el nodo de Jenkins. Por eso el pipeline pasa a `agent none` con un agente por etapa:

```
agent none
...
stage('Pruebas')          { agent { docker { image 'maven:...' } } ... }
stage('Construir imagen') { agent any   sh 'docker build --network host -t $IMAGEN .' }
stage('Smoke test')       { agent any   ... }
```

Intentar construir la imagen desde el contenedor de Maven no funcionaría: ahí no hay cliente de Docker.

```
    Pruebas                   SUCCESS    5 s
    Construir imagen          SUCCESS   39 s
    Smoke test                SUCCESS    4 s
```

El smoke test es más que un `docker build` exitoso: levanta el contenedor recién construido y le pega a `/health` hasta quince veces antes de rendirse. Si no responde, vuelca los logs y falla el build.

```
{"status":"ok"}
Imagen lista: taller-app:4
```

La imagen resultante pesa **341 MB**, y eso es mérito del `Dockerfile` multi-etapa: la primera etapa compila con `maven:3.9-eclipse-temurin-21` (unos 570 MB) y la segunda arranca de cero desde `eclipse-temurin:21-jre`, copiando solo el `.jar`. Maven, el código fuente y el repositorio de dependencias se quedan en la etapa descartada. Es exactamente la diferencia que medimos en el Trabajo en Clase 04.

Probando la imagen a mano:

```
$ docker exec jenkins curl -s 'http://docker:9000/sumar?a=2&b=3'
{"resultado":5}
```

Nótese el `http://docker:9000` y no `localhost`: desde el contenedor de Jenkins, el dind se alcanza por su alias de red `docker`. Dentro de un contenedor, `localhost` es el propio contenedor.

6. Actividad 5 — Desplegar y actualizar

La etapa final reemplaza la versión en ejecución y **verifica antes de declarar éxito**:

```
docker rm -f taller-app || true
docker run -d --name taller-app --restart unless-stopped -p 8000:8000 $IMAGEN
for i in $(seq 1 15); do
    if curl -fs http://docker:8000/version; then ... exit 0; fi
    sleep 2
done
echo "El despliegue no respondió en /version"
docker logs taller-app
exit 1
```

Parte A — la app queda corriendo.

```
Desplegada taller-app:5 en http://localhost:8000
taller-app  taller-app:5  Up 18 seconds  0.0.0.0:8000->8000/tcp

http://localhost:8001/health          -> {"status":"ok"}
http://localhost:8001/version         -> {"version":"1.0.0"}
http://localhost:8001/sumar?a=7&b=8   -> {"resultado":15}
```

Parte B — un push actualiza la versión sin intervención.

```
/version antes del push    -> {"version":"1.0.0"}
VERSION = "2.0.0"  ·  git push  ·  build #6 -> SUCCESS
/version después del build -> {"version":"2.0.0"}
contenedor en ejecución: taller-app:6
```

Parte C — un cambio roto no llega a producción. Esta es la parte que justifica todo lo anterior. Subí la versión 3.0.0 con una prueba rota a propósito:

```
build #7 -> FAILURE

  Pruebas                   FAILED
  Construir imagen          no ejecutada
  Smoke test                no ejecutada
  Desplegar                 no ejecutada

Stage "Construir imagen" skipped due to earlier failure(s)
```

```
Stage "Smoke test" skipped due to earlier failure(s)

/version sigue en          -> {"version":"2.0.0"}
contenedor en ejecución: taller-app:6  (no cambió)
```

La imagen de la 3.0.0 **nunca se construyó**, porque el pipeline se detuvo en la primera etapa. El contenedor anterior siguió atendiendo peticiones sin enterarse. Al corregir la prueba y volver a subir, el build #8 pasó y la 3.0.0 se desplegó.

Ese es el valor real del montaje: no es que automatice el despliegue, es que **automatiza la negativa a desplegar**.

7. Qué aprendí

Separar quién construye de quién ejecuta. El patrón que se repite en las cinco actividades es el mismo: Maven corre en un contenedor que se descarta, la imagen se construye en el nodo que tiene el cliente de Docker, y la app corre en el dind. Cada herramienta está donde le corresponde y ninguna contamina a las demás. Jenkins nunca tuvo Maven instalado.

`localhost` dentro de un contenedor no es lo que uno cree. Aparece tres veces en el taller y siempre por lo mismo: el smoke test usa `http://docker:18000`, el despliegue `http://docker:8000`, y la prueba manual `http://docker:9000`. Siempre el nombre del servicio, nunca `localhost`. Es la misma lección que ya me había costado en el Trabajo en Clase 03.

El pipeline es código y debe vivir con el código. Pasar de "pegar un script en Jenkins" a `Pipeline script from SCM` cambia quién es el dueño del proceso de construcción: deja de ser una configuración que alguien tocó una vez en un servidor y pasa a ser un archivo versionado, revisable y reproducible.

Una canalización sirve por lo que impide, no por lo que automatiza. Si solo desplegara más rápido, sería una comodidad. Lo que la hace valiosa es el build #7: un cambio que habría pasado desapercibido quedó detenido en la primera etapa, sin que nadie tuviera que acordarse de ejecutar las pruebas. La automatización no es solo velocidad, es la garantía de que un paso no se salta nunca.